

Language Modeling and Text Generation

MLP LAB KyungTae Lim



Contents

01 Language Modeling

02 Statistical Language Modeling

03 Computation of LM

04 Evaluation methods for LM

05 Neural Net-based LM

06 GPT, ChatGPT and Llama

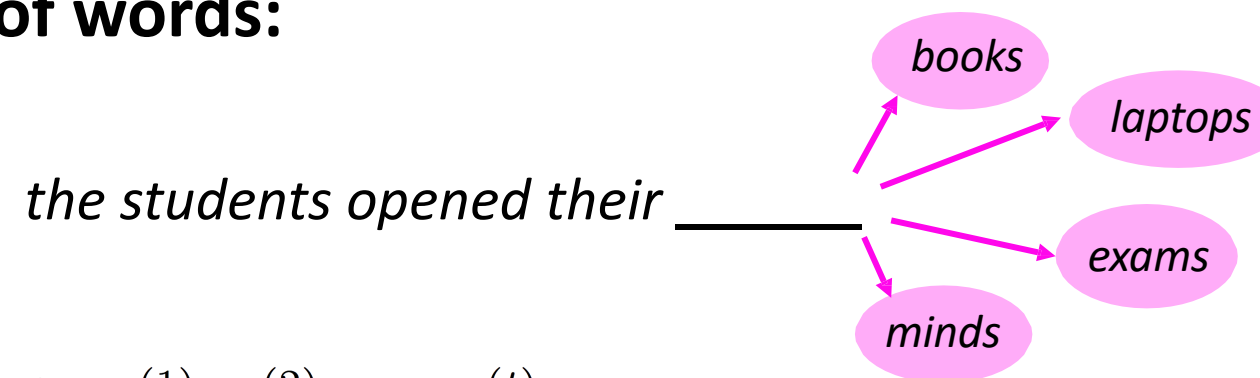


01. Language Modeling

Language Modeling

- **Language Modeling** is the task of predicting what word comes next

- For example, given a sequence of words:



- More formally: given a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$, compute the probability distribution of the next word $x^{(t+1)}$:

$$P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})$$

where $x^{(t+1)}$ can be any word in the vocabulary $V = \{w_1, \dots, w_{|V|}\}$

- A system that does this is called a **Language Model**

Language Modeling

- Language Modeling using RNN

output distribution

$$\hat{\mathbf{y}}^{(t)} = \text{softmax} \left(\mathbf{U} \mathbf{h}^{(t)} + \mathbf{b}_2 \right) \in \mathbb{R}^{|V|}$$

hidden states

$$\mathbf{h}^{(t)} = \sigma \left(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_e \mathbf{e}^{(t)} + \mathbf{b}_1 \right)$$

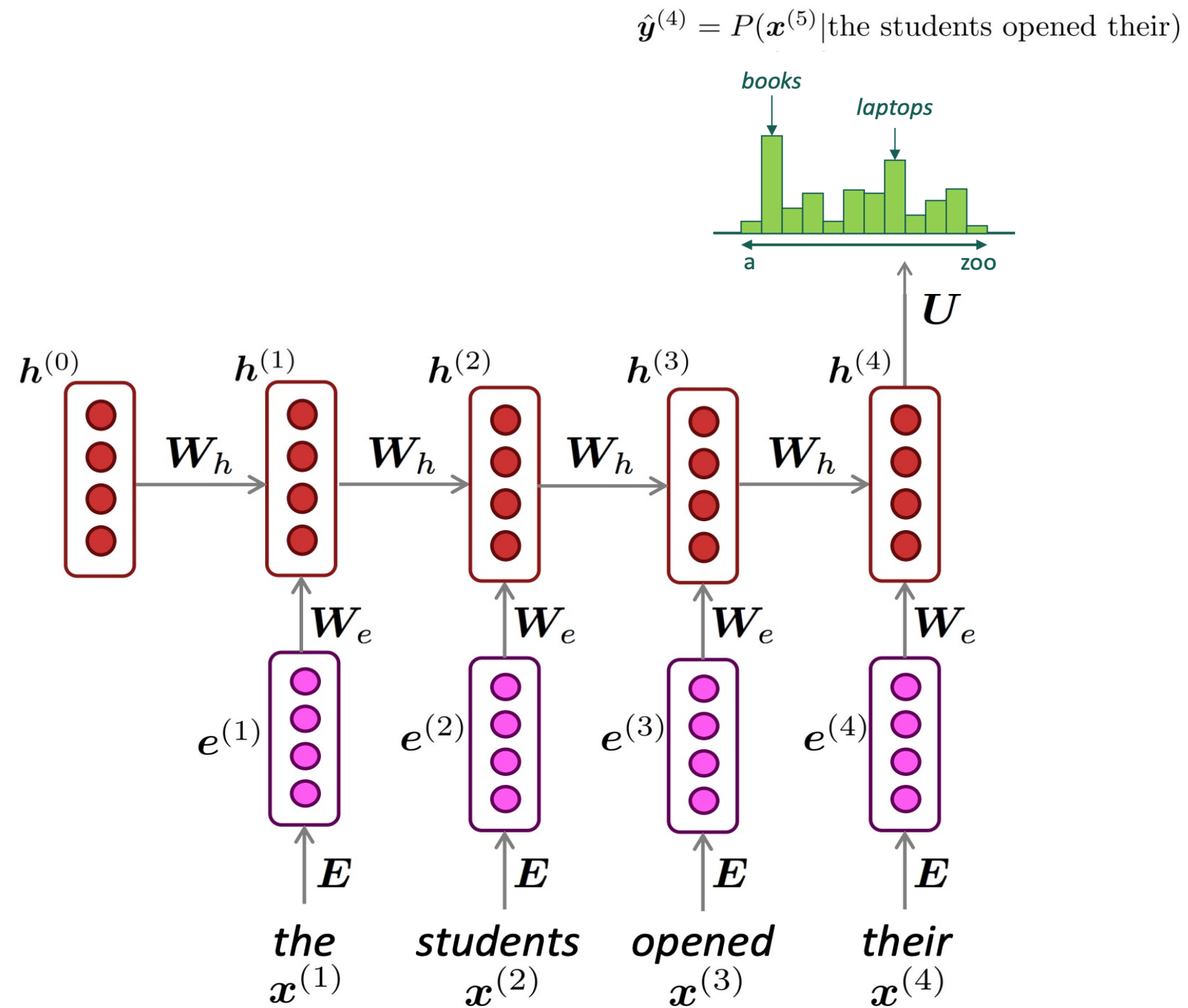
$\mathbf{h}^{(0)}$ is the initial hidden state

word embeddings

$$\mathbf{e}^{(t)} = \mathbf{E} \mathbf{x}^{(t)}$$

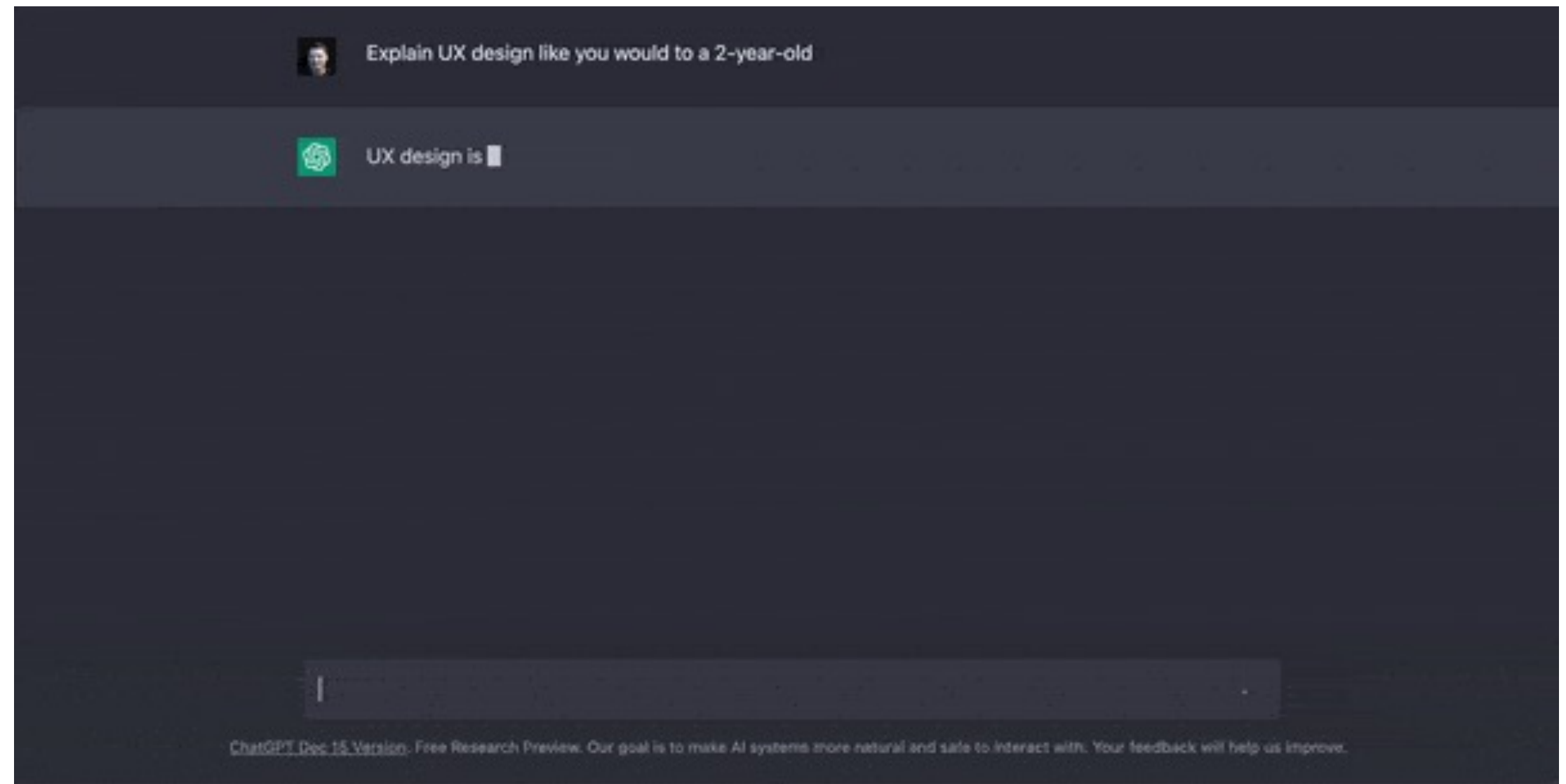
words / one-hot vectors

$$\mathbf{x}^{(t)} \in \mathbb{R}^{|V|}$$



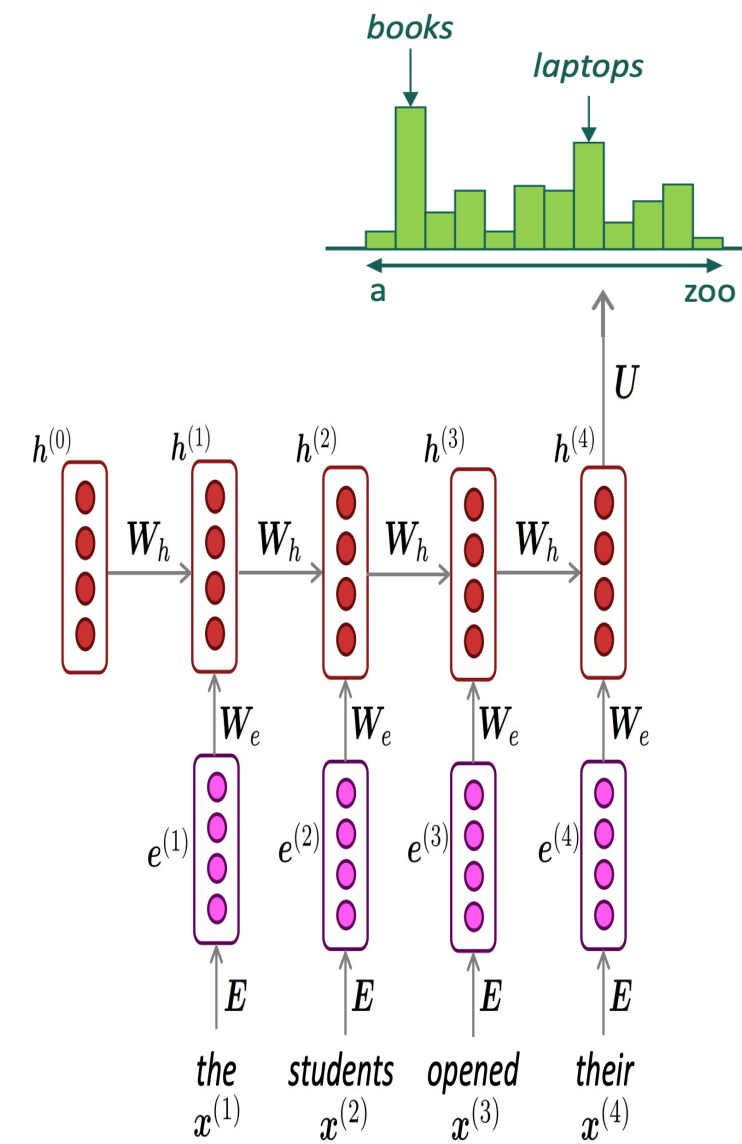
Language Modeling

- GPT??

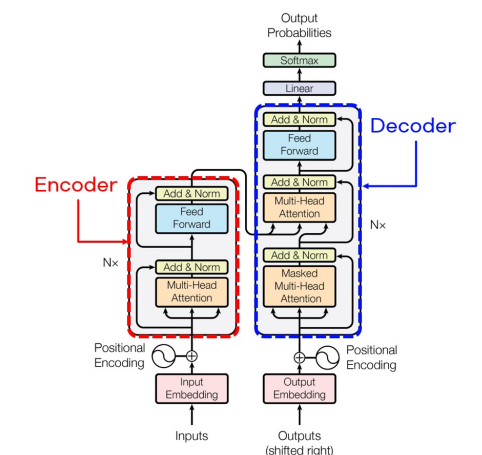
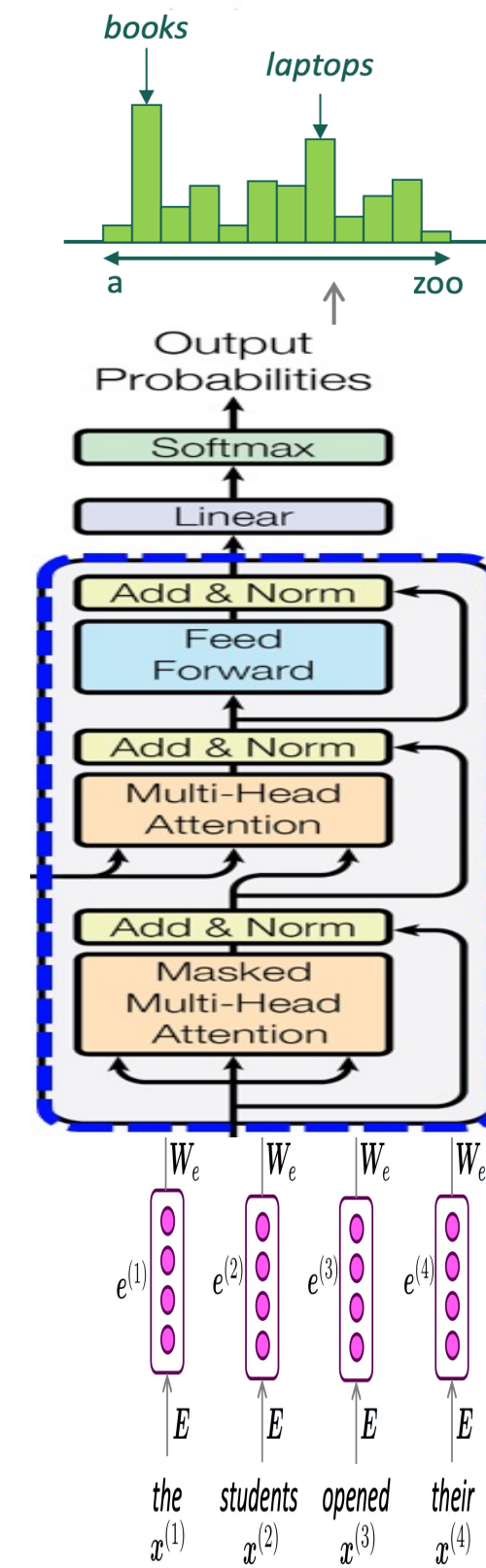


Language Modeling

- GPT is a LM trained on the decoder from Transformer



$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$



Language Modeling

- Why should we care about Language Modeling?

- Language Modeling is a **benchmark task** that helps us **measure our progress** on predicting language use
- Language Modeling is a **subcomponent** of many NLP tasks, especially those involving **generating text** or **estimating the probability of text**:
 - Predictive typing
 - Speech recognition
 - Handwriting recognition
 - Spelling/grammar correction
 - Authorship identification
 - Machine translation
 - Summarization
 - Dialogue
 - etc.
 - **Robotics**



- Everything else in NLP has now been rebuilt upon Language Modeling: **GPT is an LM!**

GPT understood the grammar and semantics of the language, so it must have generated plausible text, right?

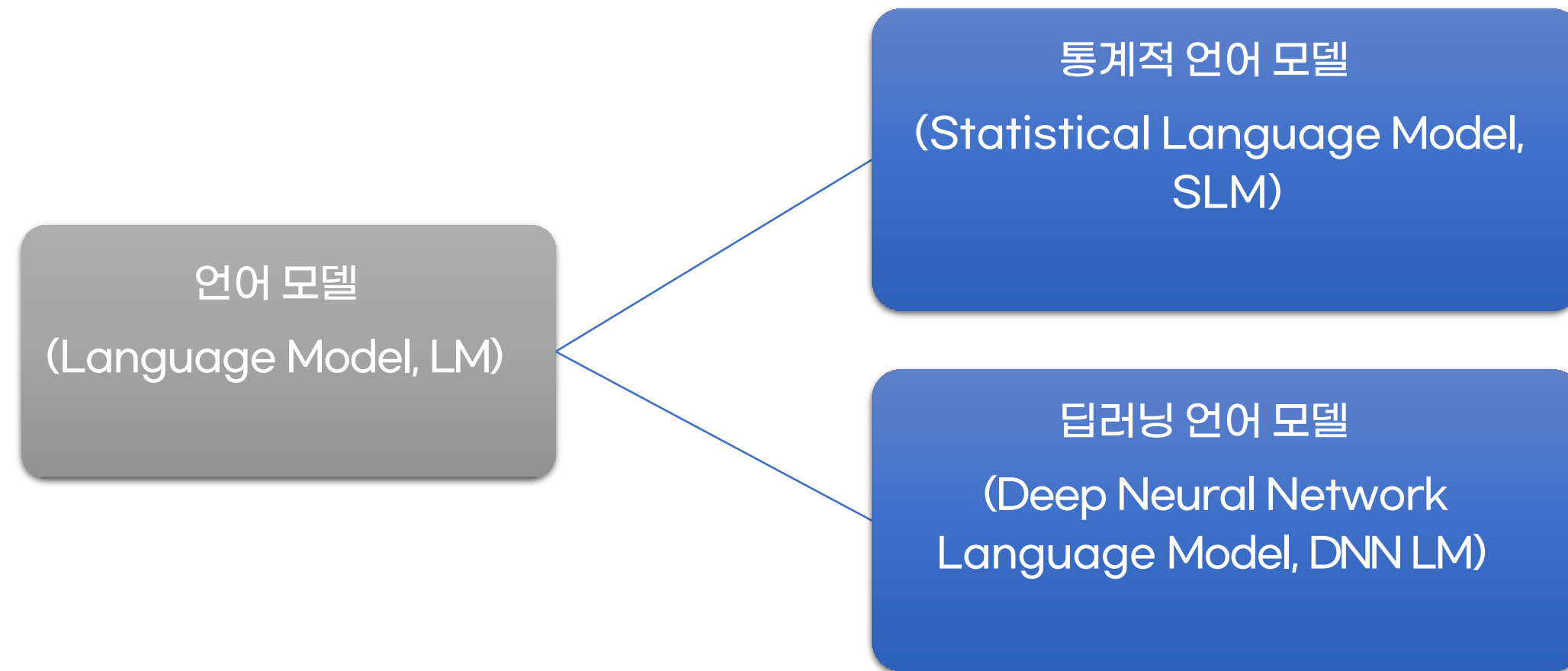
What? It understood the grammar and meaning of the language?

What the heck is a language model, how is this possible?



Language Modeling (LM)

- Now we are going to deep dive into two different types of LMs



02. Statistical Language Model

Statistical LM (SLM)

- SLM is a traditional language model that predicts combinations of words based on the probability of occurrence of a sequence of words in documents.
- The goal of SLM is to accurately approximate the probability distribution of popular word sequences.

Statistical LM (SLM)

- Conditional probability and language model
 - Conditional probability

Conditional probability **$P(B|A)$** : Probability of event B happening when event A happens



Language Modeling

Probability of word B occurring immediately after word A.

Statistical LM (SLM)

- Conditional probability and language model
 - Conditional and Combined Probabilities

$$P(B|A) = \frac{P(A, B)}{P(A)}$$

$$P(A)P(B|A) = P(A, B)$$

$$P(A, B) = P(A)P(B|A)$$

Statistical LM (SLM)

- Conditional probability and language model
 - Chain Rule

Chain rule

$$P(A, B, C, D) = P(A)P(B|A)P(C|A, B)P(D|A, B, C)$$

↓
Language Model

$$P(\text{나는, 사과를, 먹는다}) = P(\text{나는})P(\text{사과를} | \text{나는})P(\text{먹는다} | \text{나는, 사과를})$$

$$P(\text{I, eat, an, apple}) = P(\text{I}) * P(\text{eat} | \text{I}) * P(\text{an} | \text{I, eat}) * P(\text{apple} | \text{I, eat, an})$$

Statistical LM (SLM)

- We can also think of a Language Model as a system that assigns a probability to a piece of text
- For example, if we have some text, then the probability of this text (according to the Language Model) is:

Chain rule

$$P(A, B, C, D) = P(A)P(B|A)P(C|A, B)P(D|A, B, C)$$

Combination probability of n words (w)

$$P(w_1, w_2, w_3, \dots, w_n) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \dots P(w_n|w_1, \dots, w_{n-1})$$

$$= \prod_{n=1}^n P(w_n | w_1, \dots, w_{n-1})$$

This is what our LM provides

Statistical LM (SLM)

- Conditional probability and language model
 - Count-based calculations: (1) **Count** the number of times each combination of words occurs in the corpus and (2) **calculate** the probability based on that.

$$P(\text{apple} | \text{I, eat, an}) = \frac{\text{count}(\text{I, eat, an, apple})}{\text{count}(\text{I, eat, an})}$$

$$P(\text{먹는다} | \text{나는, 사과를}) = \frac{\text{count}(\text{나는, 사과를, 먹는다})}{\text{count}(\text{나는, 사과를})}$$

- Need to count the number of cases for every word combination
- Computationally complex and requires an infinitely large corpus
- Difficult, unrealistic

Statistical LM (SLM)

- Conditional probability and language model
 - Markov Assumption
 - ✓ To address and simplify the complexity of traditional chain rule
 - ✓ Assumes that a condition on a future event is independent of the past and depends only on events in the present.
 - ✓ e.g., Assume that the probability of word w_n appearing is only related to the probability of word w_{n-1} appearing before it.

$$P(w_1, w_2, \dots, w_n) \approx P(w_1)P(w_2|w_1) \cdots P(w_n|w_{n-1})$$

Statistical LM (SLM)

- Conditional probability and language model
 - Markov Assumption

$$P(w_1, w_2, \dots, w_n) \approx P(w_1)P(w_2|w_1) \cdots P(w_n|w_{n-1})$$



$$P(\text{나는, 사과를, 먹는다}) \approx P(\text{나는})P(\text{사과를} | \text{나는})P(\text{먹는다} | \text{사과를})$$

$$P(\text{I, eat, an, apple}) \approx P(\text{I}) * P(\text{eat} | \text{I}) * P(\text{an} | \text{eat}) * P(\text{apple} | \text{an})$$

Statistical LM (SLM)

- Conditional probability and language model
 - The limitation of Markov Assumption
 - ✓ **Definition:** the probability of a word's occurrence is only related to the word immediately preceding it
 - ✓ **Oversimplification** to apply to linguistic phenomena
 - ✓ **Overlooks** long-term dependencies in language
 - ✓ (e.g., The computer which I had just put into the machine room on the fifth floor crashed.)
 - ✓ Predictions may be **less accurate**

Statistical LM (SLM)

- N-gram Language

- N-gram

- ✓ Assume that a word in a sentence is associated with several words around it.
- ✓ **N**: An arbitrary number that determines how many neighboring words to look at.
- ✓ **N-gram**: A sequence of N words.
 - ✓ **1-gram(unigram)**: The, boy, is, looking, at, a, pretty, girl
 - ✓ **2-gram(bigram)**: The boy, boy is, is looking, looking at, at a, a pretty, pretty girl
 - ✓ **3-gram(trigram)**: The boy is, boy is looking, is looking at, looking at a, at a pretty, a pretty girl
 - ✓ **4-gram**: The boy is looking, boy is looking at, is looking at a, looking at a pretty, at a pretty girl

Statistical LM (SLM)

- N-gram Language Model
 - with N-1th order Markov assumptions

$$\text{1-gram(유니그램, unigram): } P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i)$$

$$\text{2-gram(바이그램, bigram): } P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i | w_{i-1})$$

...

$$\text{N-gram : } P(w_1, w_2, \dots, w_n) \approx \prod_{i=1}^n P(w_i | w_{i-N}, \dots, w_{i-1})$$

Statistical LM (SLM)

- N-gram Language Model
 - with N-1th order Markov assumptions

$$P(\text{apple} | \text{I, eat, an}) \approx P(\text{apple} | \text{an})$$

$$P(\text{먹는다} | \text{나는, 사과를}) \approx P(\text{먹는다} | \text{사과를})$$

$$\frac{\text{count}(\text{나는, 사과를, 먹는다})}{\text{count}(\text{나는, 사과를})} \approx \frac{\text{count}(\text{사과를, 먹는다})}{\text{count}(\text{사과를})}$$

$$\frac{\text{count}(\text{I, eat, an, apple})}{\text{count}(\text{I, eat, an})} \approx \frac{\text{count}(\text{an, apple})}{\text{count}(\text{an})}$$

Statistical LM (SLM)

- N-gram Language Model
 - Calculate the occurrence probability of a word string with 3 sentences and a 2-gram model

Example of dataset

<s>I eat an apple</s>

<s>an apple I eat</s>

<s>I like cheese cake</s>

Statistical LM (SLM)

■ N-gram Language Model

- Calculate the occurrence probability of a word string with 3 sentences and a 2-gram model

$$P(I|<s>) = \frac{\text{count}(<s>, I)}{\text{count}(<s>)} = \frac{2}{3} = 0.6667$$

$$P(an|<s>) = \frac{\text{count}(<s>, an)}{\text{count}(<s>)} = \frac{1}{3} = 0.3333$$

$$P(eat|I) = \frac{\text{count}(I, eat)}{\text{count}(I)} = \frac{2}{3} = 0.6667$$

$$P(</s>|apple) = \frac{\text{count}(apple, </s>)}{\text{count}(apple)} = \frac{1}{2} = 0.5$$

$$P(like|I) = \frac{\text{count}(I, like)}{\text{count}(I)} = \frac{1}{3} = 0.3333$$

$$P(cake|cheese) = \frac{\text{count}(cheese, cake)}{\text{count}(cheese)} = \frac{1}{2} = 0.5$$

Example of dataset

<s>I eat an apple</s>

<s>an apple I eat</s>

<s>I like cheese cake</s>

Statistical LM (SLM)

- N-gram Language Model
 - Example: Generate N-gram sentences based on Shakespearean works

Unigram

- ✓ To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have
- ✓ Hill he late speaks; or! A more to leg less first you enter

- 1-gram model does not consider the sequence of words and assumes that all words are independent.
- Generates words that are unrelated to each other

Statistical LM (SLM)

- N-Gram Language Model
 - Example: Generate N-gram sentences based on Shakespearean works

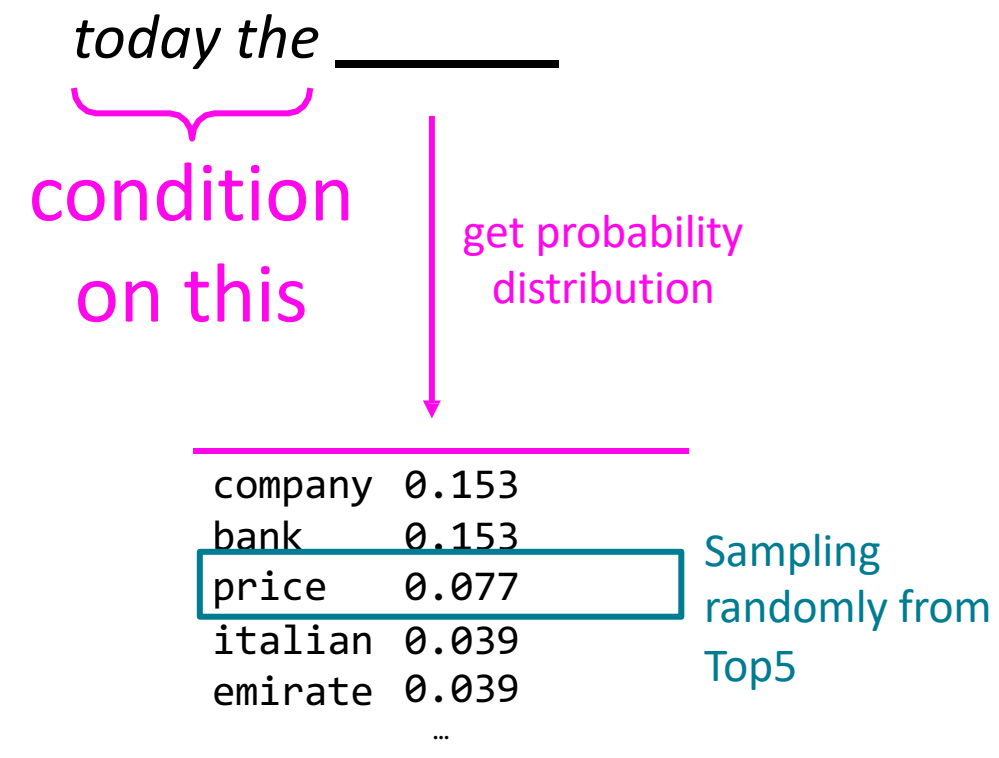
Bigram

- ✓ Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king.
Follow.
- ✓ What means, sir. I confess she? Then all sorts, he is trim, captain.

- 2-gram model considers 1 word immediately preceding it
- More natural than 1-gram, but still very unnatural overall

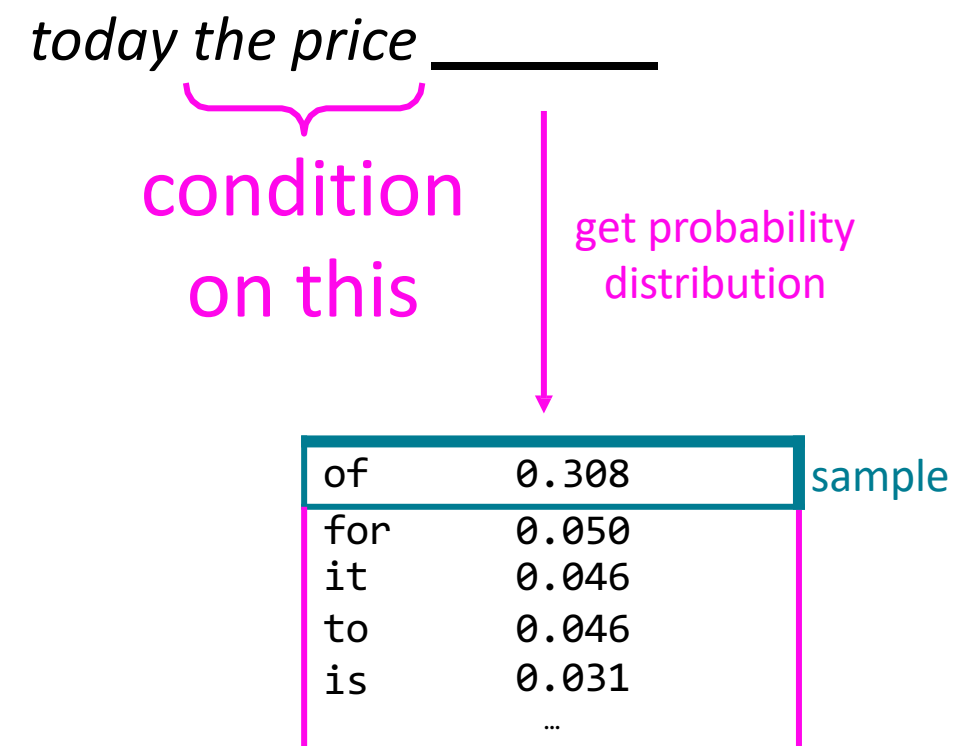
Statistical LM (SLM)

- N-gram Language Model (N-gram Language Model)
 - Example: 3-gram sentence generation computation based on Shakespearean works



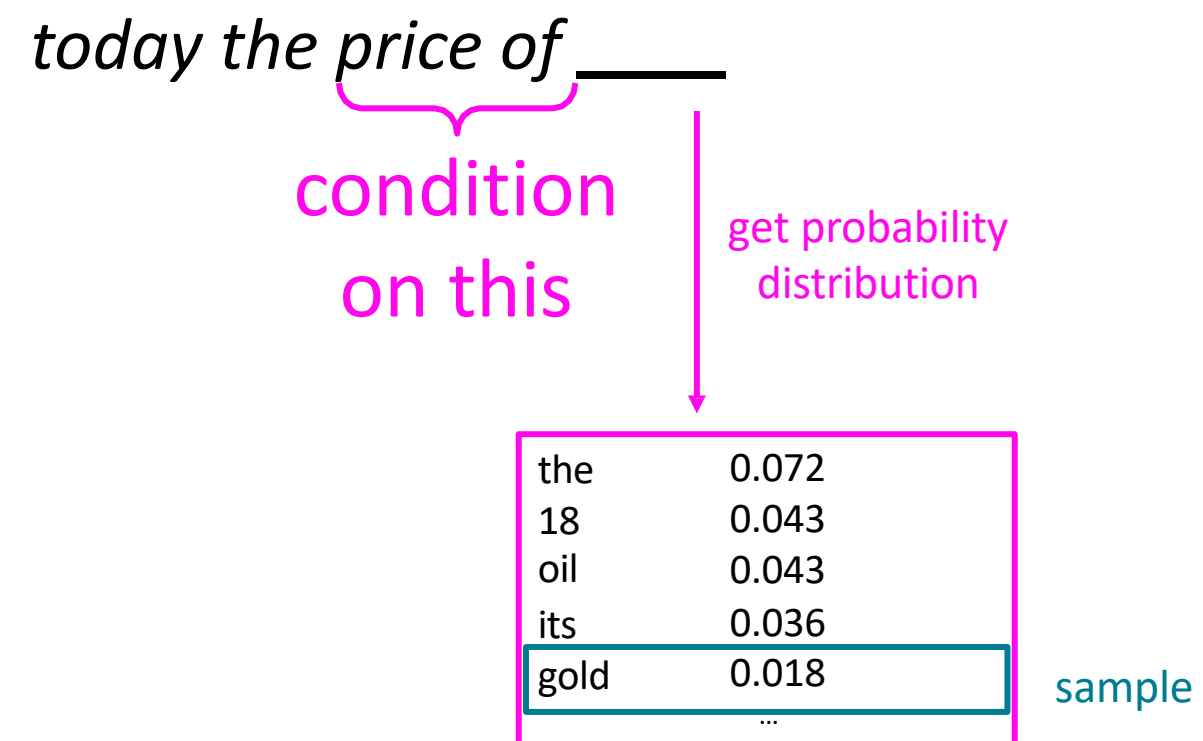
Statistical LM (SLM)

- N-gram Language Model (N-gram Language Model)
 - Example: 3-gram sentence generation computation based on Shakespearean works



Statistical LM (SLM)

- N-gram Language Model (N-gram Language Model)
 - Example: 3-gram sentence generation computation based on Shakespearean works



Statistical LM (SLM)

- N-gram Language Model (N-gram Language Model)
 - Example: 3-gram sentence generation computation based on Shakespearean works

Trigram

✓ King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

✓ It cannot be but so.

- More natural than sentences from 1-gram and 2-gram models
- Closer to the text in the corpus
- Harder to recognize as new sentences (low diversity)

- Suppose we are learning a 4-gram Language Model.

$$P(\mathbf{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \mathbf{w})}{\text{count}(\text{students opened their})}$$

- “students opened their” occurred 1000 times
- “students opened their books” occurred 400 times
 - $P(\text{books} \mid \text{students opened their}) = 0.4$
- “students opened their exams” occurred 100 times
 - $P(\text{exams} \mid \text{students opened their}) = 0.1$

Should we have discarded the “proctor” context?

Statistical LM (SLM)

- N-gram Language Model (N-gram Language Model)
 - Example: 4-gram sentence generation computation based on Shakespearean works

Generated results of 4-gram

*today the price of gold per ton , while production of shoe lasts and shoe industry ,
the bank intervened just after it considered and rejected an imf demand to rebuild
depleted european stocks , sept 30 end primary 76 cts a share .*

Surprisingly grammatical!

...but **incoherent**. We need to consider more than three words at a time if we want to model language well.

But increasing n worsens sparsity problem, and increases model size...

Statistical LM (SLM)

- N-gram Language Model (N-gram Language Model)
 - Example: 4-gram sentence generation → Sparsity Problems!!!

Sparsity Problem 1

Problem: What if “students opened their w ” never occurred in data? Then w has probability 0!

(Partial) Solution: Add small δ to the count for every $w \in V$. This is called *smoothing*.

$$P(w | \text{students opened their}) = \frac{\text{count}(\text{students opened their } w)}{\text{count}(\text{students opened their})}$$

Sparsity Problem 2

Problem: What if “students opened their” never occurred in data? Then we can’t calculate probability for *any* w !

(Partial) Solution: Just condition on “opened their” instead. This is called *backoff*.

Note: Increasing n makes sparsity problems worse. Typically, we can’t have n bigger than 5.

[추가-한국사람]

- ✓ 교착어인 한국어에서 희소성 문제가 크게 발생
- ✓ (예: 한국어에서 형태소 분석 등 전처리 진행하지 않으면 ‘사과가’, ‘사과를’, ‘사과도’, ‘사과에’는 모두 다른 단어로 처리되어 ‘사과’가 포함된 단어열의 확률값이 0에 가까워짐)

03. Computation of N-gram LM

Computation of N-gram Model

- N-gram based LM with Log probability

Markov!

$$P(w_1, w_2, w_3, \dots, w_n) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \cdots P(w_n|w_1, \dots, w_{n-1})$$

$$= \prod_{n=1}^n P(w_n|w_1, \dots, w_{n-1})$$

Log!

$$P(w_1, w_2, \dots, w_n) \approx P(w_1)P(w_2|w_1) \cdots P(w_n|w_{n-1}) \approx \prod_{i=1}^n P(w_i|w_{i-N}, \dots, w_{i-1})$$

$$P(\text{나는, 사과를, 먹는다}) \approx P(\text{나는})P(\text{사과를}|\text{나는})P(\text{먹는다}|\text{사과를})$$

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

$$\log P(w_1, w_2, \dots, w_n) \approx \sum_{i=1}^n \log P(w_i|w_{i-N}, \dots, w_{i-1})$$

- ✓ Multiplication operations require more computational resources than addition operations and have the potential to result in small values close to zero.
- ✓ Multiplication can be reduced to addition when computing the entire probability calculation in logarithmic space.
- ✓ This simplifies the calculation and avoids underflow compared to the original probability calculation.

- ✓ N-gram model expression: to compute the probability of a word string of n words in logarithmic space

Computation of N-gram Model

- Implement your own 3-gram language model!
 - ✓ **3-gram dictionaries and frequency calculations**
 - ✓ Calculating Probability Values: $P(c|a, b) = \text{3gram frequency for } (a,b,c) / \text{total number of 3grams}$
 - ✓ **Generate a sentence for the input value "There are"**

**Okay, hmm... But which of the Uni, Bi, Tri, and
Four-gram language models**



which one is the best model?

Don't we need a scale of evaluation?

Computation of N-gram Model

- Which one is better between two language models for you?

Gemini

v.s.

ChatGPT 



04. Evaluation methods for LM

Language Model Evaluation

- Evaluation of language models
 - ✓ **Human Evaluation**: Could we just have each model generate an answer to the query Q and have a human evaluate it?
 - So how long does it take to evaluate? human resources?
 - ✓ **Perplexity (PPL)**: An intrinsic evaluation measure to self-assess the performance of a language model
 - Used to evaluate models in a short amount of time, such as in a simple experiment, or as a scale for comparing models.

Language Model Evaluation

- Perplexity (Korean)

- ✓ A measure of how well a given probability model predicts a sample
- ✓ **'perplexity'**는 **'헛갈리는 정도'**를 뜻함
- ✓ 모델이 테스트 데이터셋에 대하여 확률 분포를 얼마나 확실하게 (헛갈리지 않게) 예측할 수 있는지를 나타내는 지표
- ✓ **PPL 점수가 낮을수록 (헛갈리는 경우가 적을수록) 좋음**
 - PPL이 높다는 것은 모델이 더 많은 후보군을 두고 고민하는 것
 - 예측에 대한 확실성이 낮음을 뜻함

Language Model Evaluation

- Perplexity

- ✓ A measure of how well a given probability model predicts a sample
- ✓ 'perplexity' means 'degree of confusion'
- ✓ A metric that indicates how confidently a model can predict a probability distribution over a test dataset.
- ✓ **The lower PPL score is better! (the less confusing == confidently predicted)**
 - A higher PPL means the model is considering more candidates.
 - Means less certainty about the prediction

Language Model Evaluation

- Perplexity: Which model is less confusing?
 - Indicates how many candidates the model is considering when predicting the probability of a sample.

Compare the following two models with the correct answer "gold"

Model A

today the price of _____

condition
on this

get probability
distribution

gold	0.5
the	0.3
oil	0.2
its	0.1

Model B

today the price of _____

condition
on this

get probability
distribution

gold	0.3
the	0.2
oil	0.1
Its	0.1
gas	0.1
nodap	0.1
hell	0.05
end	0.05

Language Model Evaluation

▪ Perplexity

$$PPL(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}}$$

$$= \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1, w_2, \dots, w_{i-1})}}$$

By independent assumption



If all 10 words occurred 1/10th of the time each, then

$$PPL(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}}$$

$$= \left(\frac{1}{10^N} \right)^{-\frac{1}{N}}$$

$$= \left(\frac{1}{10} \right)^{-1}$$

$$= 10$$

✓ **N-gram models enforce the cascading law under the 'N-1st order Markov assumption'**

✓ To reflect this, the formula for the 2-gram model is as follows

$$PPL(W) = \sqrt[N]{\frac{1}{\prod_{i=1}^N P(w_i | w_{i-1})}}$$

Language Model Evaluation

▪ Perplexity

- The standard **evaluation metric** for Language Models is **perplexity**.

$$\text{perplexity} = \prod_{t=1}^T \left(\frac{1}{P_{\text{LM}}(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})} \right)^{1/T}$$

Inverse probability of corpus, according to Language Model

Normalized by
number of words

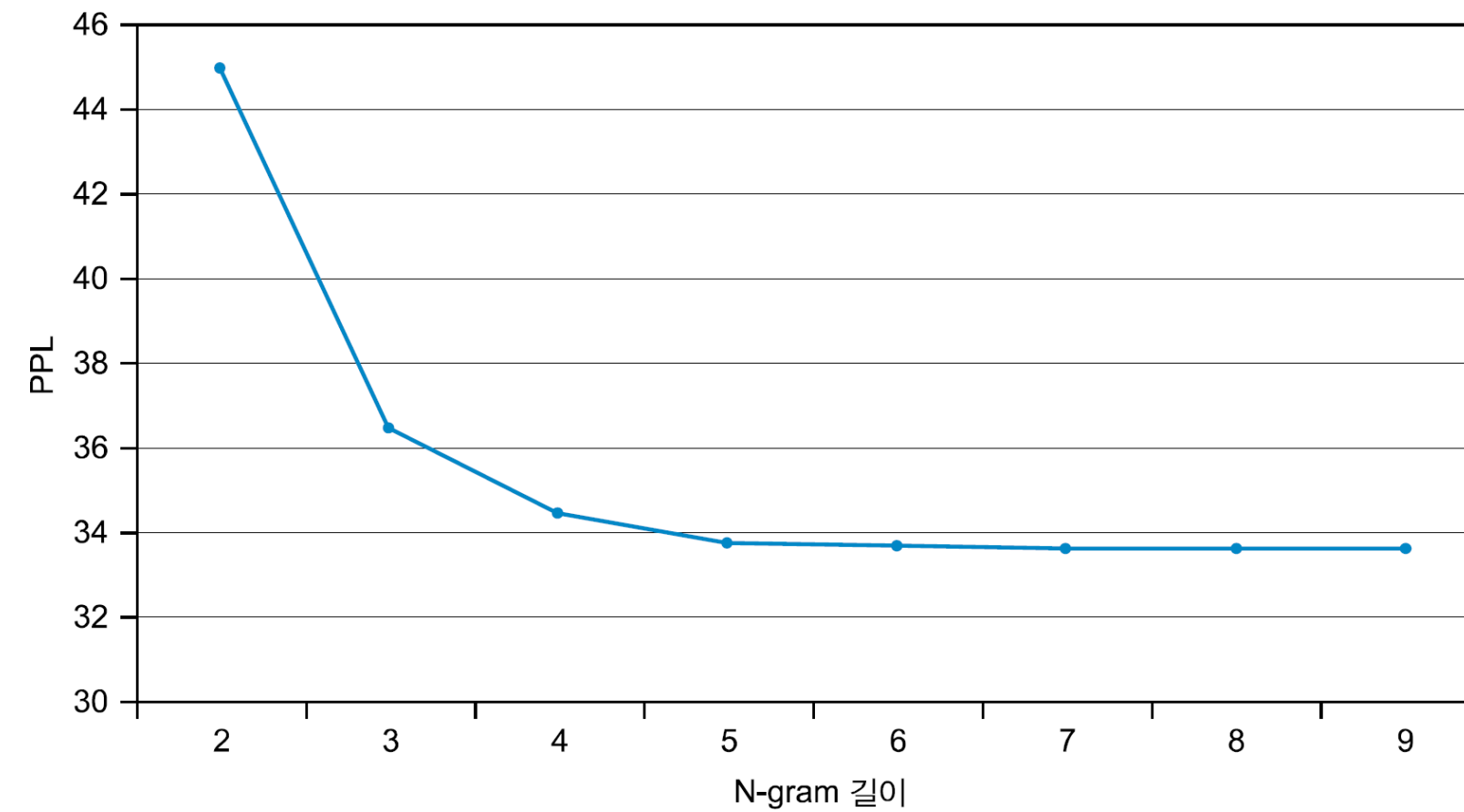
- This is equal to the exponential of the cross-entropy loss $J(\theta)$:

$$= \prod_{t=1}^T \left(\frac{1}{\hat{y}_{\mathbf{x}_{t+1}}^{(t)}} \right)^{1/T} = \exp \left(\frac{1}{T} \sum_{t=1}^T -\log \hat{y}_{\mathbf{x}_{t+1}}^{(t)} \right) = \exp(J(\theta))$$

Lower perplexity is better!

Language Model Evaluation

- Compare the performance of N-gram language models

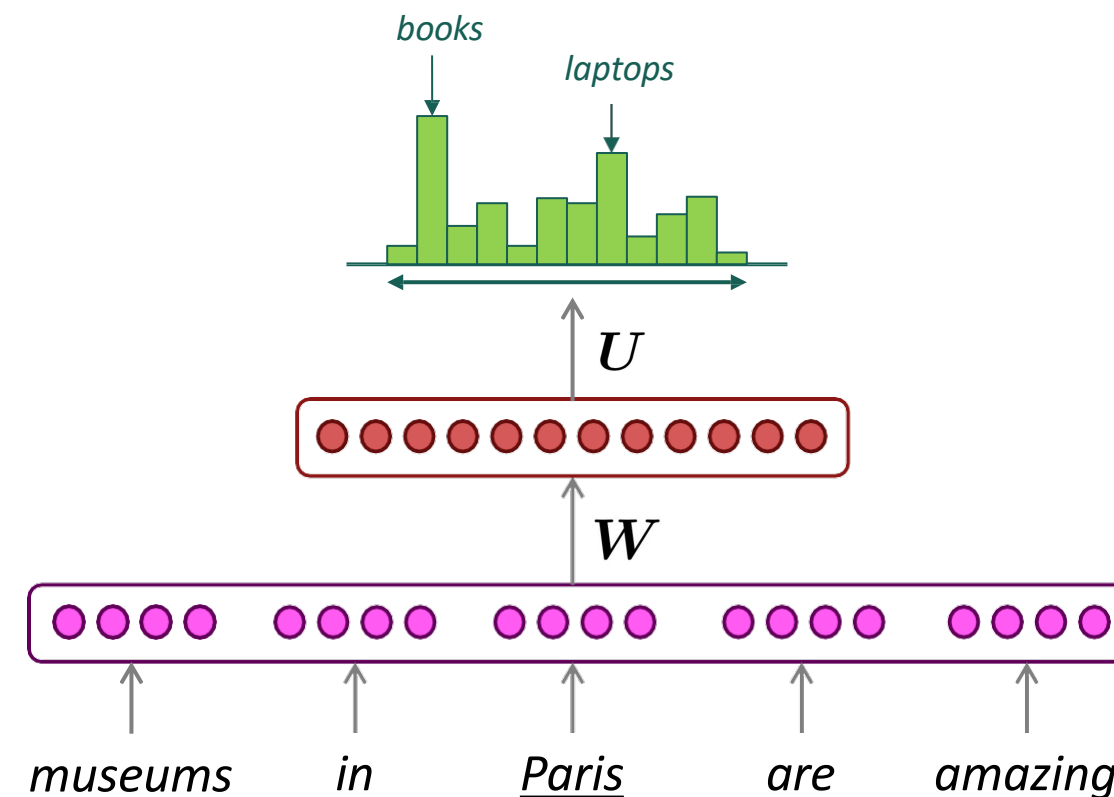


05. Neural Net-based LM

How to build a neural language model?

- Fixed-window Neural LM: 5-gram with FFNN LM model

- Recall the Language Modeling task:
 - Input: sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$
 - Output: prob. dist. of the next word $P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})$
- How about a [window-based neural model](#)?



A fixed-window neural language Model

- 5-gram with FFNN LM model

output distribution

$$\hat{y} = \text{softmax}(U\mathbf{h} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

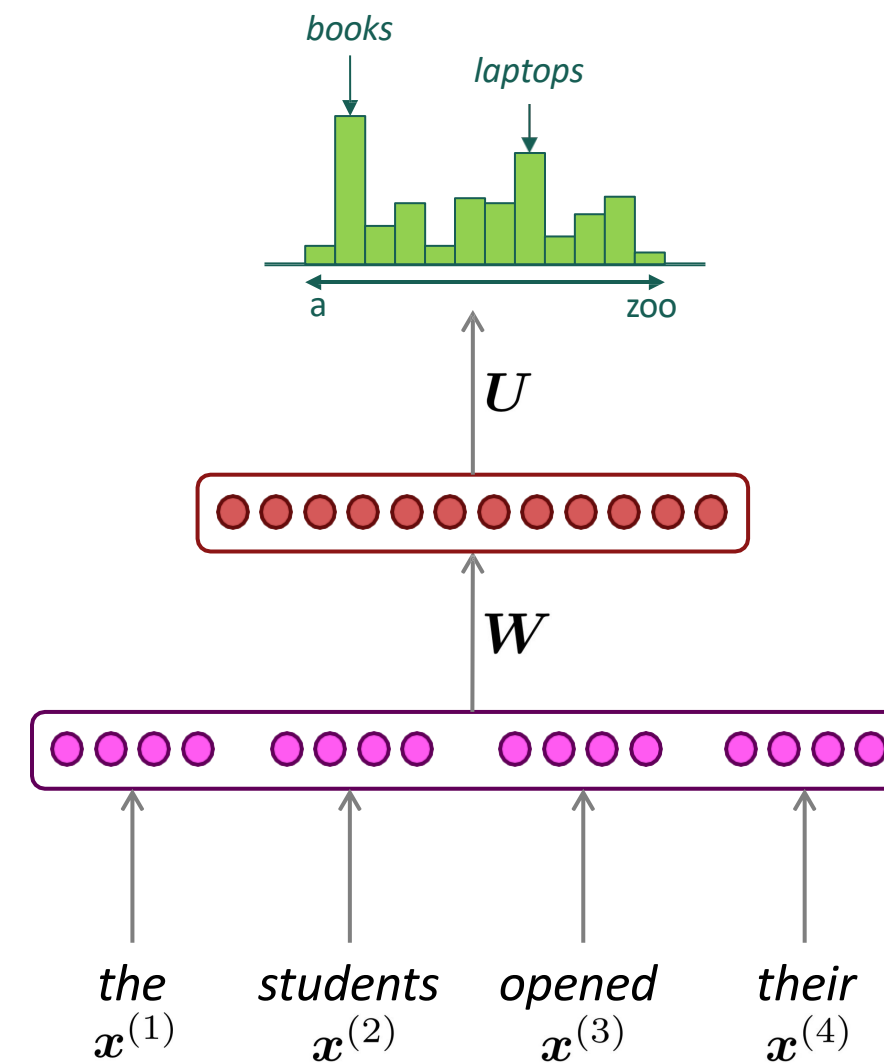
$$\mathbf{h} = f(W\mathbf{e} + \mathbf{b}_1)$$

concatenated word embeddings

$$\mathbf{e} = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

words / one-hot vectors

$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \mathbf{x}^{(4)}$$



Computation of N-gram Model

- Implement your own 5-gram with FFNN LM model
 - ✓ **Build a word dictionary!**
 - ✓ **Prepare your corpus on 5-gram**
 - ✓ Code your FFNN model
 - ✓ **Generate a sentence for the input value "There are"**
 - ✓ If you want, you can use a pretrained word embedding!

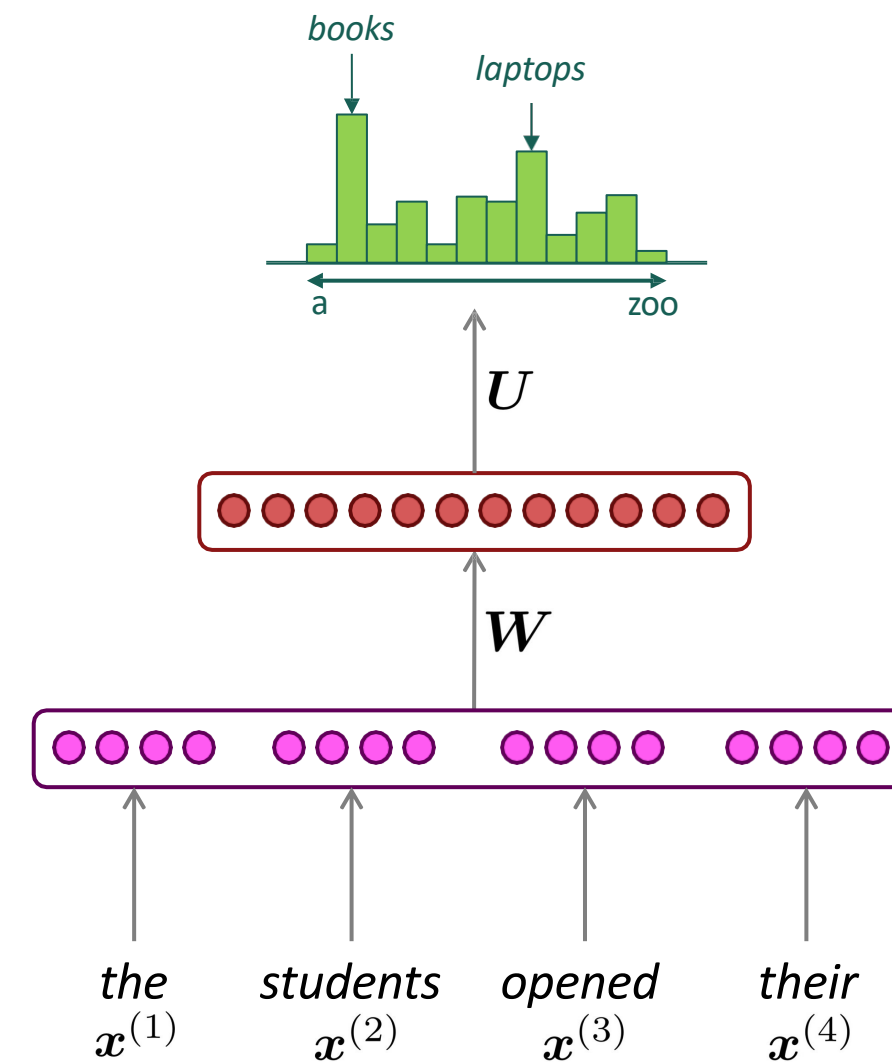
A fixed-window neural language Model

- 5-gram with FFNN LM model

Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

Improvements over n -gram LM:

- No sparsity problem
- Don't need to store all observed n -grams



A fixed-window neural language Model

▪ Critical problems of 5-gram with FFNN LM model

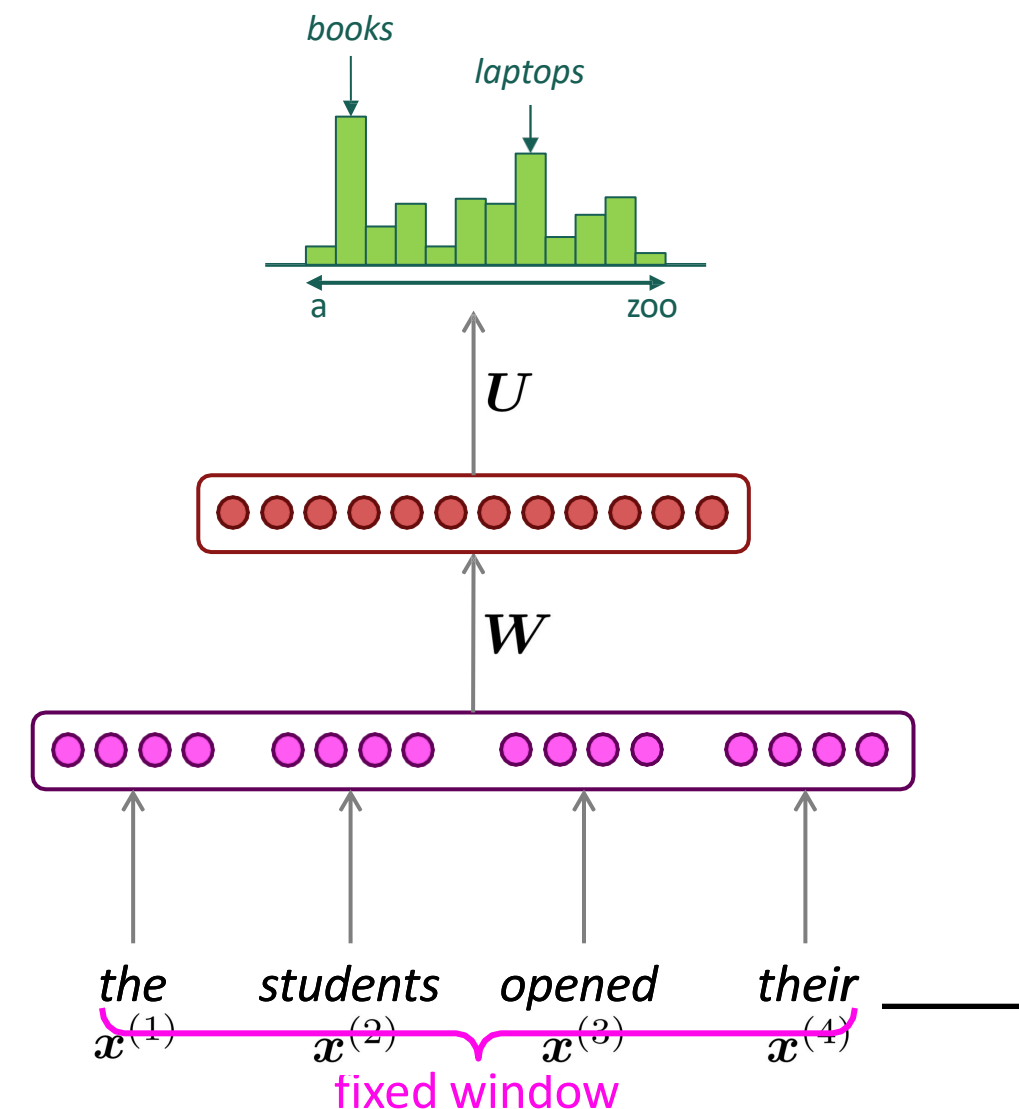
Remaining **problems**:

- Fixed window is **too small**
- Enlarging window enlarges W
- Window can never be large enough!
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weights in W .

No symmetry in how the inputs are processed.

We need a neural architecture that can process *any length input*

~~as the proctor started the clock~~
discard



A fixed-window neural language Model

- The problem of the 5-gram with FFNN LM model

We need a neural architecture
that can process *any length input*

A **Non**-fixed-window neural language Model

- RNN-based LM model

output distribution

$$\hat{y}^{(t)} = \text{softmax} \left(U h^{(t)} + b_2 \right) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right)$$

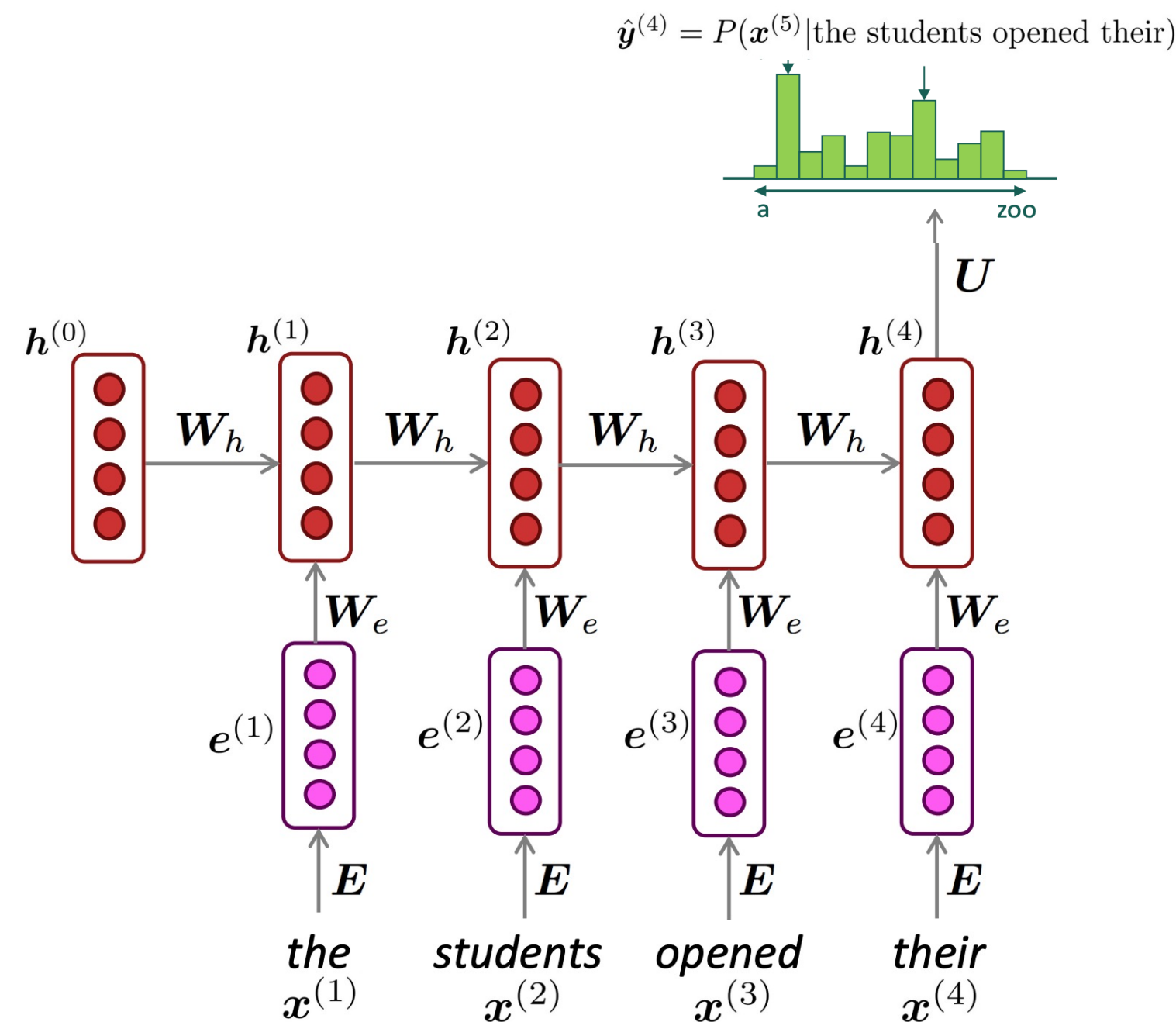
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Note: this input sequence could be much longer now!

A **Non**-fixed-window neural language Model

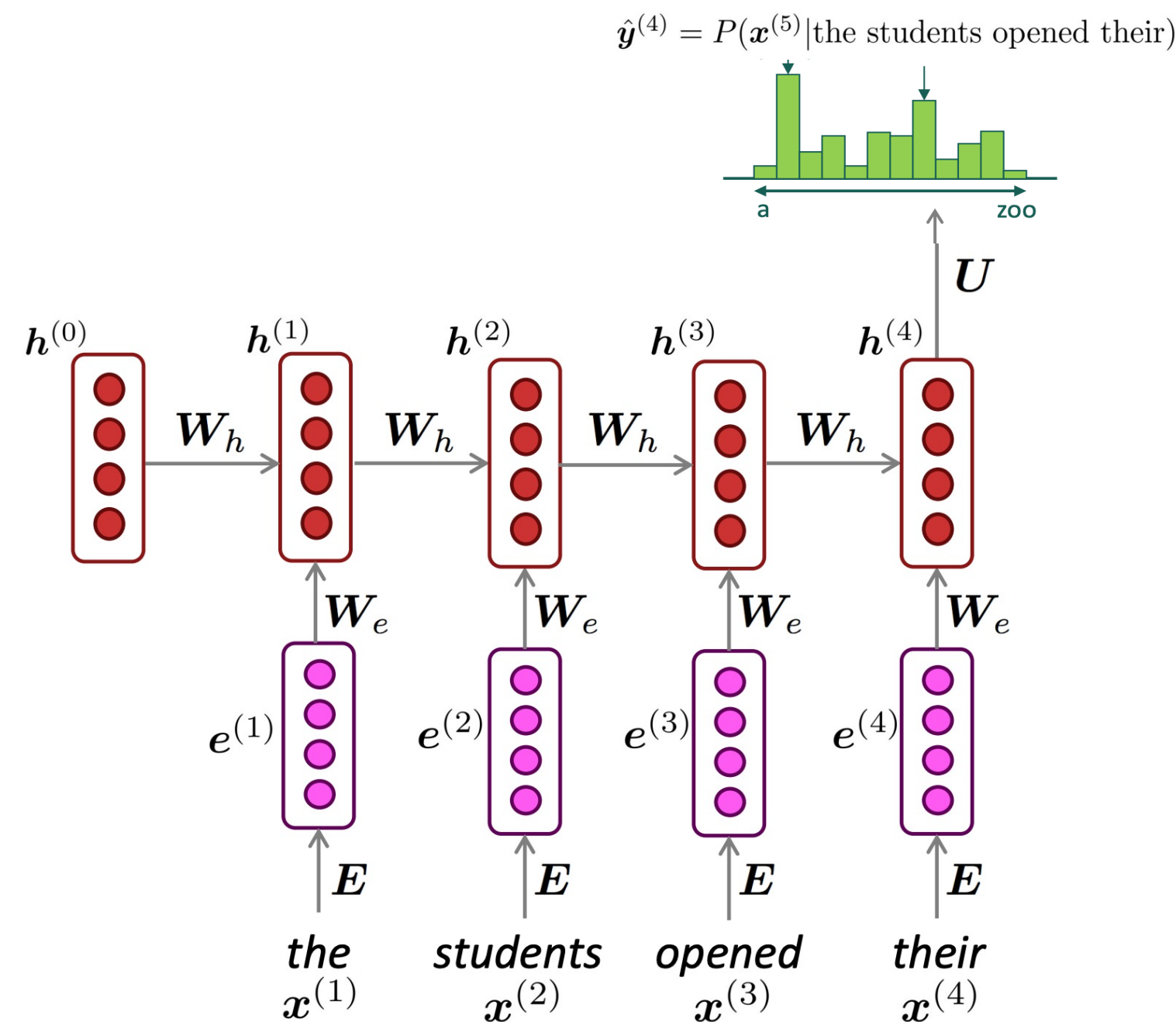
▪ RNN-based LM model

RNN **Advantages**:

- Can process **any length** input
- Computation for step t can (in theory) use information from **many steps back**
- **Model size doesn't increase** for longer input context
- Same weights applied on every timestep, so there is **symmetry** in how inputs are processed.

RNN **Disadvantages**:

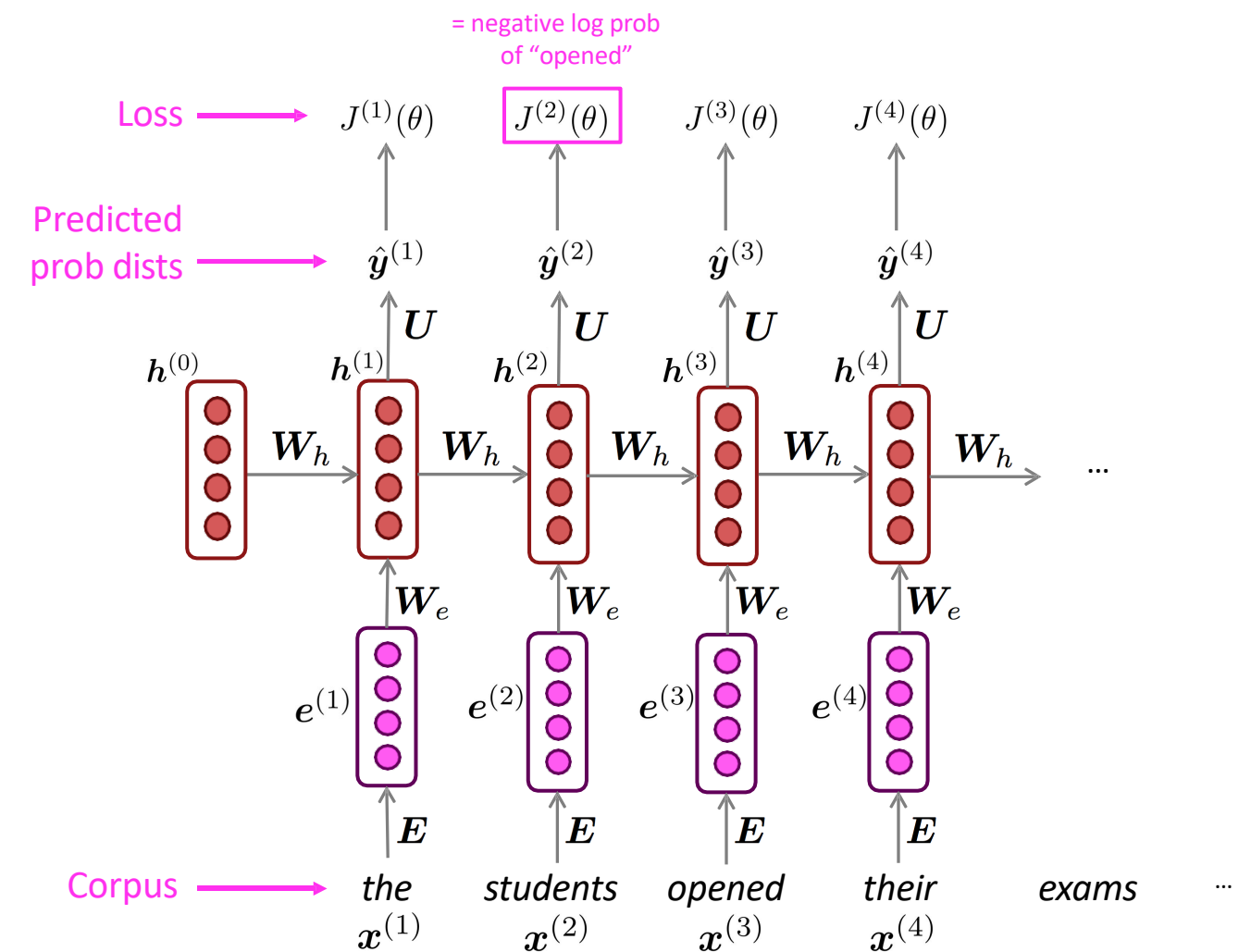
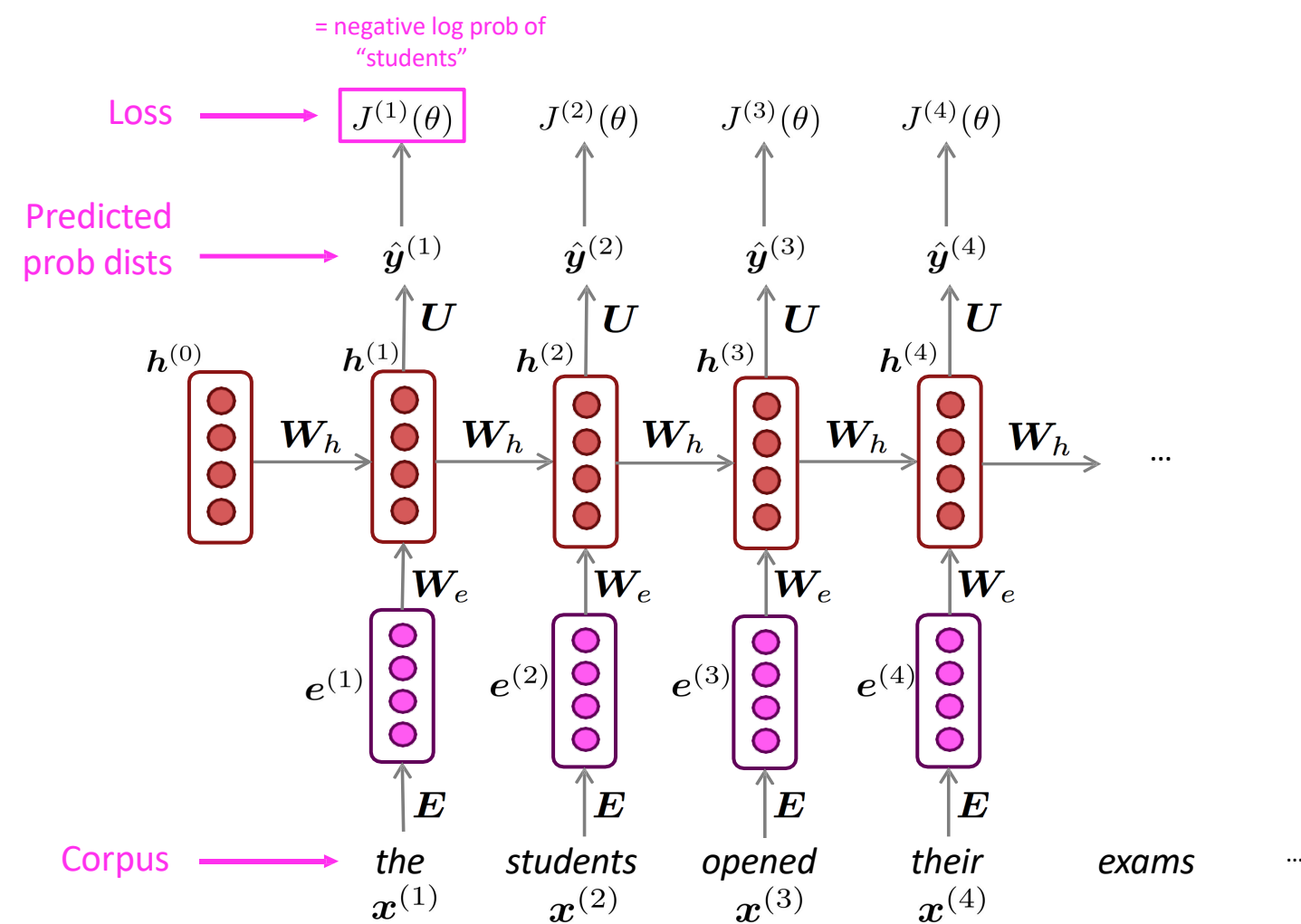
- Recurrent computation is **slow**
 - In practice, difficult to access information from **many steps back**
- More on these later



Note: this input sequence could be much longer now!

A **Non**-fixed-window neural language Model

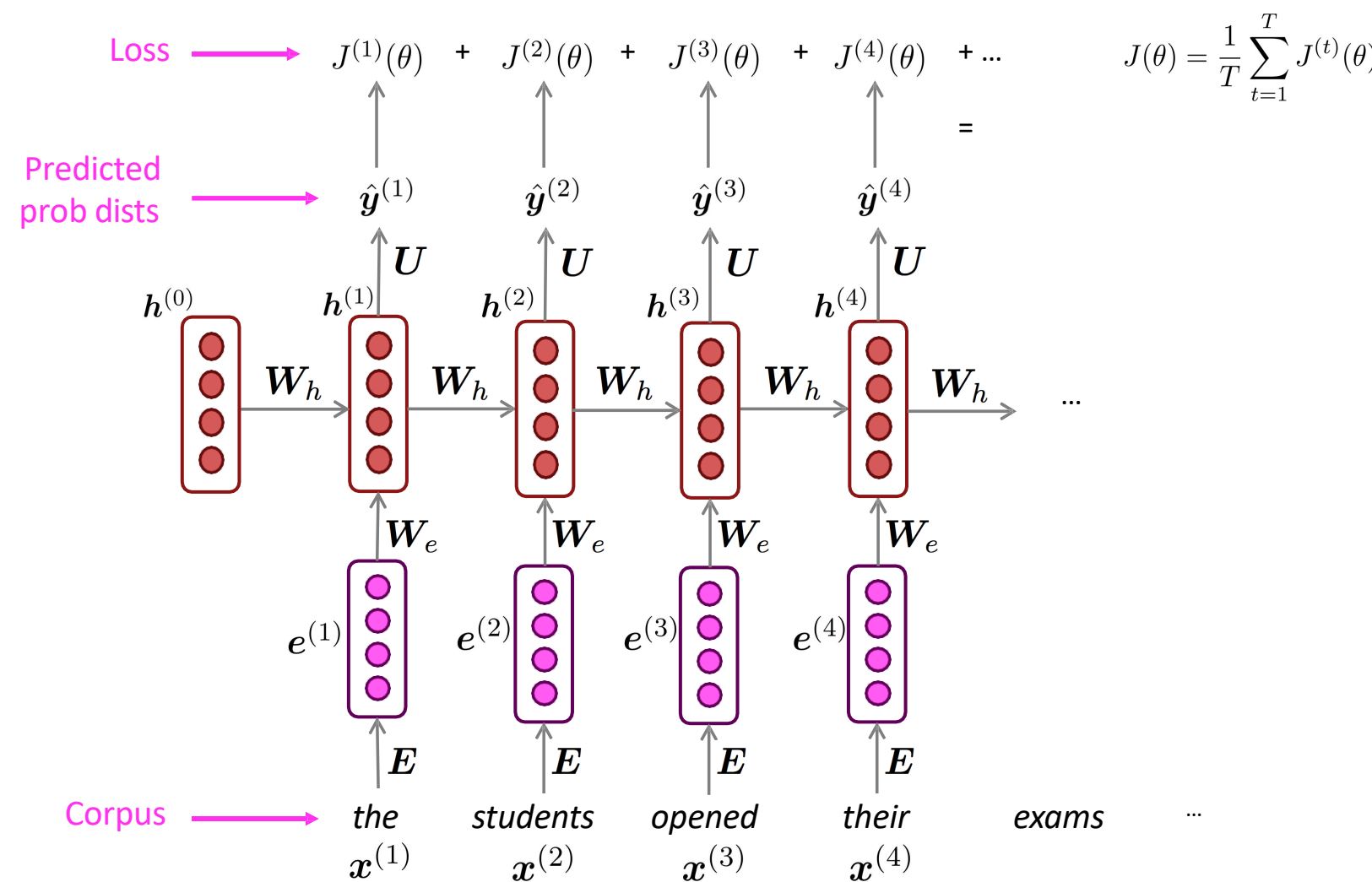
- RNN-based LM model



A **Non**-fixed-window neural language Model

▪ RNN-based LM model

“Teacher forcing”



- Get a **big corpus of text** which is a sequence of words $x^{(1)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ **for every step t** .
 - i.e., predict probability dist of *every word*, given words so far
- **Loss function** on step t is **cross-entropy** between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE(y^{(t)}, \hat{y}^{(t)}) = - \sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)} = - \log \hat{y}_{x_{t+1}}^{(t)}$$

- Average this to get **overall loss** for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{y}_{x_{t+1}}^{(t)}$$

A **Non**-fixed-window neural language Model

- RNN-based LM model

Let's have some fun!

- You can train an RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on **Obama speeches**:



The United States will step up to the cost of a new challenges of the American people that will share the fact that we created the problem. They were attacked and so that they have to say that all the task of the final days of war that I will not be able to get this done.

Source: <https://medium.com/@samim/obama-rnn-machine-generated-political-speeches-c8abd18a2ea0>

A **Non**-fixed-window neural language Model

▪ RNN-based LM model

Let's have some fun!

- You can train an RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on *Harry Potter*:



“Sorry,” Harry shouted, panicking—“I’ll leave those brooms in London, are they?”

“No idea,” said Nearly Headless Nick, casting low close by Cedric, carrying the last bit of treacle Charms, from Harry’s shoulder, and to answer him the common room perched upon it, four arms held a shining knob from when the spider hadn’t felt it seemed. He reached the teams too.

Source: <https://medium.com/deep-writing/harry-potter-written-by-artificial-intelligence-8a9431803da6>

A **Non**-fixed-window neural language Model

▪ RNN-based LM model

Let's have some fun!

- You can train an RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on **recipes**:

Title: CHOCOLATE RANCH BARBECUE
Categories: Game, Casseroles, Cookies, Cookies
Yield: 6 Servings

2 tb Parmesan cheese -- chopped
1 c Coconut milk
3 Eggs, beaten

Place each pasta over layers of lumps. Shape mixture into the moderate oven and simmer until firm. Serve hot in bodied fresh, mustard, orange and cheese.

Combine the cheese and salt together the dough in a large skillet; add the ingredients and stir in the chocolate and pepper.

Source: <https://gist.github.com/nylki/1efbaa36635956d35bcc>



Language Model Evaluation

- N-gram VS Neural Net LM

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

n-gram model

Increasingly complex RNNs

Perplexity improves (lower is better)

Source: <https://research.fb.com/building-an-efficient-neural-language-model-over-a-billion-words/>

06. GPT, ChatGPT, Llama, DeepSeek

We will discuss later...

Thank you!